

Practical 6: Component Demo - Shopping List App

Duration: 2 hours

Learning Objectives

- Integrate everything from Labs 1–5 into a single small application.
- Practise combining layout, state, navigation, and lists without new concepts.

Prerequisites

Labs 1–5 completed.

Lab Tasks

1. Choose one of the provided briefs (e.g. a simple shopping list, a habit tracker, or a contacts directory).
2. Plan two or three screens and sketch the navigation graph before coding.
3. Build a list screen using LazyColumn populated from in-memory sample data.
4. Add an "add item" screen with TextFields and state; pass the new item back or append it to the list.
5. Add a detail screen reached by tapping a list row.
6. Polish layout and spacing, and confirm the app survives basic navigation back and forth.

Key Concepts

- LazyColumn — A vertically scrolling list that composes only visible items.
- ExposedDropdownMenuBox — A Material 3 dropdown anchored to a read-only text field. The expanded/collapsed state is a plain Boolean held in state.
- Checkbox / RadioButton — Selection controls. Checkboxes are independent on/off toggles; radio buttons form a group where exactly one option is selected.
- AlertDialog — A pop-up dialog. It is shown or hidden purely by state: when a state variable is set, the dialog composable is emitted; when cleared, it disappears.
- State hoisting — Keeping the item list in a parent composable above the NavHost so both screens read and write the same list.

Dependencies needed (add this in gradle Module:app level)

```
dependencies {  
    implementation("androidx.core:core-ktx:1.13.1")  
    implementation("androidx.activity:activity-compose:1.9.0")  
    implementation(platform("androidx.compose:compose-bom:2024.06.00"))  
    implementation("androidx.compose.ui:ui")  
    implementation("androidx.compose.ui:ui-graphics")  
    implementation("androidx.compose.ui:ui-tooling-preview")  
    implementation("androidx.compose.material3:material3")  
    implementation("androidx.navigation:navigation-compose:2.7.7")  
}
```

Lab Tasks

1. Create a new Empty Activity (Compose) project and name it **Practical_6_<yourID>**. For this project change the target sdk to 37 and the compile version to 37, then add the dependencies listed above and sync Gradle.
2. Add a package directory: select your **com.example.<projectname>** package, right click and select New -> Package. Name it **com.example.<yourProjectName>.model**. Inside it create a new Kotlin Class/File, choose Data Class, and name it **ShoppingItem**. This data class holds one shopping list entry, including the category (chosen from the dropdown), the priority (chosen with radio buttons) and two Boolean flags used by the checkboxes.

```
package com.example.practical_6_amit3353.model
```

```
data class ShoppingItem(  
    val id: Int,  
    val name: String,  
    val category: String,  
    val priority: String,  
    val urgent: Boolean,  
    val purchased: Boolean = false  
)
```

3. Add another package named **data**. Create a new Kotlin Class/File (Object) named **ShoppingData**. It holds the fixed choices for the dropdown (categories) and the radio buttons (priorities), plus the sample items that populate the list at startup. Import the ShoppingItem data class.

```
package com.example.practical_6_amit3353.data
```

```
import com.example.practical_6_amit3353.model.ShoppingItem
```

```
object ShoppingData {
```

```
    val categories = listOf("Groceries", "Household", "Electronics", "Other")
```

```
    val priorities = listOf("Low", "Medium", "High")
```

```
    val sampleItems = listOf(  
        ShoppingItem(1, "Milk", "Groceries", "High", urgent = true),  
        ShoppingItem(2, "Light bulbs", "Household", "Low", urgent = false),  
        ShoppingItem(3, "USB cable", "Electronics", "Medium", urgent = false),  
        ShoppingItem(4, "Bread", "Groceries", "Medium", urgent = false)  
    )
```

```
}
```

4. Add a new package called **screen**. Create a new Kotlin Class/File and name it **ShoppingItemRow**. Each row demonstrates a **Checkbox**: ticking it marks the item as purchased and draws a strikethrough on the name. The delete icon does not delete immediately — it only reports the click upward so the list screen can show a confirmation dialog first.

```
package com.example.practical_6_amit3353.screen
```

```
import androidx.compose.foundation.layout.Column  
import androidx.compose.foundation.layout.Row  
import androidx.compose.foundation.layout.fillMaxWidth  
import androidx.compose.foundation.layout.padding  
import androidx.compose.material.icons.Icons  
import androidx.compose.material.icons.filled.Delete  
import androidx.compose.material3.Card
```

```

import com.example.practical_6_amit3353.model.ShoppingItem

@Composable
fun ShoppingItemRow(
    item: ShoppingItem,
    onCheckedChange: (Boolean) -> Unit,
    onDeleteClick: () -> Unit
) {
    Card(modifier = Modifier.fillMaxWidth()) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 8.dp, vertical = 4.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Checkbox: tick off purchased items
            Checkbox(
                checked = item.purchased,
                onCheckedChange = onCheckedChange
            )
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = item.name,
                    style = MaterialTheme.typography.titleMedium,
                    textDecoration =
                        if (item.purchased) TextDecoration.LineThrough else null
                )
                Text(
                    text = buildString {
                        append(item.category)
                        append(" - ")
                        append(item.priority)
                        if (item.urgent) append(" - URGENT")
                    },
                    style = MaterialTheme.typography.bodySmall
                )
            }
            IconButton(onClick = onDeleteClick) {
                Icon(Icons.Filled.Delete, contentDescription = "Delete item")
            }
        }
    }
}

```

5. In the same **screen** package, create **ListScreen**. This screen demonstrates the **LazyColumn** and the **AlertDialog**. Note the dialog pattern: a nullable state variable (`itemPendingDelete`) controls visibility. Tapping a row's delete icon sets it; Cancel or tapping outside sets it back to null; Delete removes the item and then clears it.

```

package com.example.practical_6_amit3353.screen

import com.example.practical_6_amit3353.model.ShoppingItem

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ListScreen(
    items: List<ShoppingItem>,
    onAddClick: () -> Unit,
    onTogglePurchased: (ShoppingItem) -> Unit,
    onDelete: (ShoppingItem) -> Unit
) {
    // Dialog state: when this is not null, the confirm dialog is showing
    var itemPendingDelete by remember { mutableStateOf<ShoppingItem?>(null) }
}

```

```

Scaffold(
  topBar = { TopAppBar(title = { Text("Shopping List Demo") }) },
  floatingActionButton = {
    FloatingActionButton(onClick = onAddClick) {
      Icon(Icons.Filled.Add, contentDescription = "Add item")
    }
  }
) { innerPadding ->
  if (items.isEmpty()) {
    Box(
      modifier = Modifier
        .fillMaxSize()
        .padding(innerPadding),
      contentAlignment = Alignment.Center
    ) {
      Text("No items yet. Tap + to add one.")
    }
  } else {
    // LazyColumn: composes only the rows that are visible
    LazyColumn(
      modifier = Modifier
        .fillMaxSize()
        .padding(innerPadding),
      contentPadding = PaddingValues(16.dp),
      verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
      items(items, key = { it.id }) { item ->
        ShoppingItemRow(
          item = item,
          onCheckedChange = { onTogglePurchased(item) },
          onDeleteClick = { itemPendingDelete = item }
        )
      }
    }
  }
}

// AlertDialog: confirm before deleting
itemPendingDelete?.let { item ->
  AlertDialog(
    onDismissRequest = { itemPendingDelete = null },
    title = { Text("Delete item?") },
    text = {
      Text("Are you sure you want to delete " + item.name + "?")
    },
    confirmButton = {
      TextButton(onClick = {
        onDelete(item)
        itemPendingDelete = null
      }) { Text("Delete") }
    },
    dismissButton = {
      TextButton(onClick = { itemPendingDelete = null }) {
        Text("Cancel")
      }
    }
  )
}
}

```

6. Still in the **screen** package, create **AddItemScreen**. This screen demonstrates the **dropdown list** (ExposedDropdownMenuBox), the **RadioButton group** and a labelled **Checkbox**. Note three details: (a) the dropdown's text field is readOnly and must have .menuAnchor() or the menu will not open; (b) the radio buttons pass onClick = null and let the whole Row handle

the click via the selectable modifier with `Role.RadioButton` — this makes the entire row tappable and accessible; (c) the same Row + selectable pattern is used for the urgent checkbox.

```
package com.example.practical_6_amit3353.screen

import com.example.practical_6_amit3353.data.ShoppingData

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AddItemScreen(
    onSave: (name: String, category: String, priority: String, urgent: Boolean) -> Unit,
    onBack: () -> Unit
) {
    var name by remember { mutableStateOf("") }
    var selectedCategory by remember { mutableStateOf(ShoppingData.categories.first()) }
    var dropdownExpanded by remember { mutableStateOf(false) }
    var selectedPriority by remember { mutableStateOf(ShoppingData.priorities[1]) }
    var urgent by remember { mutableStateOf(false) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Add Item") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(
                            Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Back"
                        )
                    }
                }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            // Text input
            OutlinedTextField(
                value = name,
                onValueChange = { name = it },
                label = { Text("Item name") },
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )

            // Dropdown list (ExposedDropdownMenuBox)
            ExposedDropdownMenuBox(
                expanded = dropdownExpanded,
                onExpandedChange = { dropdownExpanded = it }
            ) {
                OutlinedTextField(
                    value = selectedCategory,
                    onValueChange = {},
                    readOnly = true,
                    label = { Text("Category") },
                    trailingIcon = {
                        ExposedDropdownMenuDefaults.TrailingIcon(
                            expanded = dropdownExpanded
                        )
                    }
                )
            }
        }
    }
}
```

```

        modifier = Modifier
            .fillMaxWidth()
            .menuAnchor()
    )
    ExposedDropdownMenu(
        expanded = dropdownExpanded,
        onDismissRequest = { dropdownExpanded = false }
    ) {
        ShoppingData.categories.forEach { category ->
            DropdownMenuItem(
                text = { Text(category) },
                onClick = {
                    selectedCategory = category
                    dropdownExpanded = false
                }
            )
        }
    }
}

// RadioButton group
Text("Priority", style = MaterialTheme.typography.titleSmall)
Column {
    ShoppingData.priorities.forEach { priority ->
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .selectable(
                    selected = (priority == selectedPriority),
                    onClick = { selectedPriority = priority },
                    role = Role.RadioButton
                )
            .padding(vertical = 4.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            RadioButton(
                selected = (priority == selectedPriority),
                onClick = null // click handled by the Row
            )
            Spacer(Modifier.width(8.dp))
            Text(priority)
        }
    }
}

// Checkbox with label
Row(
    modifier = Modifier
        .fillMaxWidth()
        .selectable(
            selected = urgent,
            onClick = { urgent = !urgent },
            role = Role.Checkbox
        )
    .padding(vertical = 4.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    Checkbox(checked = urgent, onCheckedChange = null)
    Spacer(Modifier.width(8.dp))
    Text("Mark as urgent")
}

Spacer(Modifier.height(8.dp))

Button(

```

```

        onClick = {
            onSave(name.trim(), selectedCategory, selectedPriority, urgent)
        },
        enabled = name.isNotBlank(),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Save")
    }
}
}
}
}

```

7. Add a new package and name it **nav**. Create a Kotlin Class/File named **AppNavGraph**. The item list is created here with `mutableStateListOf`, above the `NavHost`, so both screens share the same list and items added on the Add screen are still present after navigating back (state hoisting). The graph has two routes: "list" (start destination) and "add".

```

package com.example.practical_6_amit3353.nav

import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.remember
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.example.practical_6_amit3353.data.ShoppingData
import com.example.practical_6_amit3353.model.ShoppingItem
import com.example.practical_6_amit3353.screen.AddItemScreen
import com.example.practical_6_amit3353.screen.ListScreen

@Composable
fun AppNavGraph(navController: NavHostController = rememberNavController()) {

    // Hoisted list state: shared by both screens, survives navigation
    val items = remember {
        mutableStateListOf<ShoppingItem>().apply { addAll(ShoppingData.sampleItems) }
    }

    NavHost(navController = navController, startDestination = "list") {

        composable("list") {
            ListScreen(
                items = items,
                onAddClick = { navController.navigate("add") },
                onTogglePurchased = { item ->
                    val index = items.indexOfFirst { it.id == item.id }
                    if (index != -1) {
                        items[index] =
                            items[index].copy(purchased = !items[index].purchased)
                    }
                },
                onDelete = { item -> items.removeAll { it.id == item.id } }
            )
        }

        composable("add") {
            AddItemScreen(
                onSave = { name, category, priority, urgent ->
                    val nextId = (items.maxOfOrNull { it.id } ?: 0) + 1
                    items.add(
                        ShoppingItem(
                            id = nextId,

```

```

        name = name,
        category = category,
        priority = priority,
        urgent = urgent
    )
    )
    navController.popBackStack()
},
onBack = { navController.popBackStack() }
)
}
}
}

```

8. In your **MainActivity**, call the **AppNavGraph** and run your project.

```

setContent {
    MaterialTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            AppNavGraph()
        }
    }
}

```

9. Test the full flow: scroll the list, tick Milk off (strikethrough appears), tap + to open the Add screen, type a name, pick a category from the dropdown, select a priority, tick Mark as urgent, press Save. Back on the list, the new item appears with its category, priority and URGENT tag. Tap its delete icon, press Cancel first (item remains), then delete again and press Delete to confirm removal.

Deliverable

Submit / Demonstrate

A small multi-screen app combining a list, input with state, and navigation.

Challenge (optional extension)

Make the added items persist within the session by hoisting the list state to a shared parent so it is not lost on navigation.

Common Pitfalls

Watch out for

In-memory data resets when the app process restarts — that is expected this week; persistence comes next.

Keep scope small. A polished three-screen app beats an unfinished ambitious one.